

MYSQL COMPLETE PROJECT NOTES

MySQL Shell → SQL Mode → Assignment 5 → Queries → Aggregate / GROUP BY / HAVING

Purpose: A practical, command-focused reference based on the SQL work studied in the project and the supplied Assignment 5 materials. No unnecessary theory/story.

1. Project Flow: MySQL Shell se SQL tak

The project workflow starts from MySQL Shell and then switches into SQL mode. The earlier project work used the sequence mysqlsh → \sql → connection/database selection → SQL queries.

```
mysqlsh
\sql
\connect root@localhost
\use rrgi
```

Important: MySQL Shell commands such as \sql, \connect, \use and \system cls are Shell commands, not normal SQL statements; they are used at the MySQL Shell prompt.

```
\system cls
```

Use \system cls to clear the Windows terminal screen. It clears only the visible terminal; it does not delete database/table/data.

After entering SQL mode, SQL statements are entered at a prompt such as MySQL localhost:33060+ ssl ... SQL >. SQL statements normally end with ;.

2. Database / Table Inspection Commands

```
SHOW DATABASES;
USE rrgi;
SHOW TABLES;
DESC employee;
DESCRIBE employee;
```

DESC / DESCRIBE: shows column name, data type, NULL information, key, default and extra details.

3. Employee Table Used in the Practical

```
CREATE TABLE employee(
emp_id INT PRIMARY KEY,
emp_name VARCHAR(45),
emp_age INT,
emp_department VARCHAR(30),
emp_salary INT,
emp_city VARCHAR(45)
);
```

The supplied practical output confirms these six columns and emp_id as the primary key.

4. Insert 10 Employees

```
INSERT INTO employee
VALUES
(101, 'Aditya', 19, 'IT', 60000, 'Delhi'),
(102, 'Aman', 20, 'HR', 65000, 'Delhi'),
```

```
(103, 'Arun', 22, 'IT', 68000, 'Lucknow'),  
(104, 'Anil', 21, 'CSE', 70000, 'Hydrabad'),  
(105, 'Saurabh', 21, 'CSD', 960000, 'Sitapur'),  
(106, 'Shivam', 21, 'IT', 899999, 'Jaunpur'),  
(107, 'Krishna', 22, 'HR', 78000, 'Lucknow'),  
(108, 'Adarsh', 22, 'CSE', 69000, 'Delhi'),  
(109, 'Chandan', 21, 'IT', 80000, 'Ballia'),  
(110, 'Sunil', 21, 'IT', 90000, 'Jaunpur');
```

Safer form: explicitly write the column names when inserting data.

```
INSERT INTO employee  
(emp_id, emp_name, emp_age, emp_department, emp_salary, emp_city)  
VALUES  
(111, 'Rahul', 22, 'IT', 35000, 'Gorakhpur');
```

Primary-key rule: inserting an already-used emp_id causes a duplicate-primary-key error. The project demonstrated this with emp_id 101.

5. Basic CRUD Commands

CREATE

```
CREATE TABLE employee (...);
```

READ / SELECT

```
SELECT * FROM employee;  
SELECT emp_name, emp_salary FROM employee;
```

UPDATE

```
UPDATE employee  
SET emp_salary = 10000000  
WHERE emp_name = 'Saurabh';
```

```
UPDATE employee  
SET emp_name = 'XYZ'  
WHERE emp_id = 104;
```

DELETE

```
DELETE FROM employee WHERE emp_id = 110;  
DELETE FROM employee WHERE emp_name = 'Aditya';
```

Golden rule: UPDATE/DELETE me WHERE condition check karo. WHERE chhootne par unwanted multiple/all rows change/delete ho sakte hain.

6. Assignment 5 — Question-wise Solutions

The supplied Assignment 5 asks first for table creation, 10 inserts, display/update/delete tasks, then filtering, sorting, grouping and aggregate-function questions. [filecite turn1file2 L2-L15](#)

Q1. Find employees earning more than 50,000.

```
SELECT * FROM employee WHERE emp_salary > 50000;
```

Q2. Find employees from IT.

```
SELECT * FROM employee WHERE emp_department = 'IT';
```

Q3. Find employees from Lucknow.

```
SELECT * FROM employee WHERE emp_city = 'Lucknow';
```

Q4. Find employees earning between 40,000 and 60,000.

```
SELECT * FROM employee WHERE emp_salary BETWEEN 40000 AND 60000;
```

Q5. Find employees whose name starts with A.

```
SELECT * FROM employee WHERE emp_name LIKE 'A%';
```

Q6. Find employees whose name ends with a.

```
SELECT * FROM employee WHERE emp_name LIKE '%a';
```

Q7. Find employees belonging to IT or HR.

```
SELECT * FROM employee WHERE emp_department = 'IT' OR emp_department = 'HR';
```

Q8. Find employees older than 25 and earning more than 50,000.

```
SELECT * FROM employee WHERE emp_age > 25 AND emp_salary > 50000;
```

Q9. Find employees whose salary is NULL.

```
SELECT * FROM employee WHERE emp_salary IS NULL;
```

Q10. Find employees whose salary is not NULL.

```
SELECT * FROM employee WHERE emp_salary IS NOT NULL;
```

Q11. Sort employees by salary.

```
SELECT * FROM employee ORDER BY emp_salary ASC;
```

Q12. Sort employees from highest salary to lowest.

```
SELECT * FROM employee ORDER BY emp_salary DESC;
```

Q13. Count employees in every department.

```
SELECT emp_department, COUNT(emp_name) AS Number_Of_emp  
FROM employee  
GROUP BY emp_department;
```

Q14. Find average salary per department.

```
SELECT emp_department, AVG(emp_salary) AS avg_salary  
FROM employee  
GROUP BY emp_department;
```

Q15. Find maximum salary per department.

```
SELECT emp_department, MAX(emp_salary) AS max_salary  
FROM employee  
GROUP BY emp_department;
```

Q16. Find departments having more than 5 employees.

```
SELECT emp_department  
FROM employee
```

```
GROUP BY emp_department  
HAVING COUNT(emp_name) > 5;
```

Q17. Find departments whose average salary is greater than 60,000.

```
SELECT emp_department  
FROM employee  
GROUP BY emp_department  
HAVING AVG(emp_salary) > 60000;
```

Q18. Sort departments by average salary.

```
SELECT emp_department, AVG(emp_salary) AS avg_salary  
FROM employee  
GROUP BY emp_department  
ORDER BY AVG(emp_salary) ASC;
```

Q19. Find the number of employees in every city.

```
SELECT emp_city, COUNT(emp_name) AS Number_Of_emp  
FROM employee  
GROUP BY emp_city;
```

Q20. Find cities having more than 3 employees.

```
SELECT emp_city  
FROM employee  
GROUP BY emp_city  
HAVING COUNT(emp_name) > 3;
```

Q21. Find total employees.

```
SELECT COUNT(*) AS total_employees FROM employee;
```

Q22. Find total salary.

```
SELECT SUM(emp_salary) AS total_salary FROM employee;
```

Q23. Find average salary.

```
SELECT AVG(emp_salary) AS average_salary FROM employee;
```

Q24. Find maximum salary.

```
SELECT MAX(emp_salary) AS maximum_salary FROM employee;
```

Q25. Find minimum salary.

```
SELECT MIN(emp_salary) AS minimum_salary FROM employee;
```

Q26. Find average salary of IT employees.

```
SELECT AVG(emp_salary) AS avg_salary  
FROM employee  
WHERE emp_department = 'IT';
```

Q27. Find highest salary in HR.

```
SELECT MAX(emp_salary) AS highest_salary  
FROM employee  
WHERE emp_department = 'HR';
```

Q28. Find total salary paid to Finance employees.

```
SELECT SUM(emp_salary) AS total_salary  
FROM employee  
WHERE emp_department = 'Finance';
```

Q29. Find number of employees in Delhi.

```
SELECT COUNT(*) AS Number_Of_emp  
FROM employee  
WHERE emp_city = 'Delhi';
```

Q30. Find average salary of employees earning more than 50,000.

```
SELECT AVG(emp_salary) AS avg_salary
FROM employee
WHERE emp_salary > 50000;
```

Q31. Find department-wise total salary.

```
SELECT emp_department, SUM(emp_salary) AS total_salary
FROM employee
GROUP BY emp_department;
```

Q32. Find department-wise average salary.

```
SELECT emp_department, AVG(emp_salary) AS avg_salary
FROM employee
GROUP BY emp_department;
```

Q33. Find department with the highest average salary.

```
SELECT emp_department, AVG(emp_salary) AS avg_salary
FROM employee
GROUP BY emp_department
ORDER BY AVG(emp_salary) DESC
LIMIT 1;
```

Q34. Find department with the highest total salary.

```
SELECT emp_department, SUM(emp_salary) AS total_salary
FROM employee
GROUP BY emp_department
ORDER BY SUM(emp_salary) DESC
LIMIT 1;
```

Q35. Find city-wise employee count.

```
SELECT emp_city, COUNT(emp_name) AS Number_Of_emp
FROM employee
GROUP BY emp_city;
```

7. Project me Actually Practiced Query Versions

The corrected Assignment 5 solution supplied with the project uses some practical-specific thresholds. These are kept separately so that the original assignment wording and the practiced version do not get mixed.

Salary > 70,000

```
SELECT * FROM employee WHERE emp_salary > 70000;
```

Salary strictly between 70,000 and 90,000

```
SELECT * FROM employee  
WHERE emp_salary > 70000  
AND emp_salary < 90000;
```

Age > 21 AND salary > 70,000

```
SELECT * FROM employee  
WHERE emp_age > 21  
AND emp_salary > 70000;
```

Departments having more than 2 employees

```
SELECT emp_department  
FROM employee  
GROUP BY emp_department  
HAVING COUNT(emp_name) > 2;
```

Departments with average salary > 70,000

```
SELECT emp_department  
FROM employee  
GROUP BY emp_department  
HAVING AVG(emp_salary) > 70000;
```

Average salary by department, ascending

```
SELECT emp_department, AVG(emp_salary) AS avg_salary  
FROM employee  
GROUP BY emp_department  
ORDER BY AVG(emp_salary) ASC;
```

Cities having more than 1 employee

```
SELECT emp_city  
FROM employee  
GROUP BY emp_city  
HAVING COUNT(emp_name) > 1;
```

8. Most Important Rules & Common Errors

1) INSERT syntax

```
INSERT INTO employee VALUES (...);
```

Wrong pattern seen in practice: INSERT INTO employee (values...). Correct form is INSERT INTO employee VALUES (...).

2) UPDATE syntax

```
UPDATE employee SET emp_salary = 10000000 WHERE emp_name = 'Saurabh';
```

Wrong: UPDATE FROM employee Correct: UPDATE employee SET

3) LIKE vs =

```
SELECT * FROM employee WHERE emp_name LIKE 'A%';
```

Use LIKE for patterns. % means any number of characters; _ means one character.

4) OR condition

```
WHERE emp_department = 'IT'  
OR emp_department = 'HR';
```

Do not write WHERE emp_department = 'IT' OR 'HR'. Repeat the column comparison, or use IN ('IT','HR').

5) NULL

```
WHERE emp_salary IS NULL;  
WHERE emp_salary IS NOT NULL;
```

Do not use = NULL for NULL checking.

6) GROUP BY column must match the selected non-aggregate column

```
SELECT emp_city, COUNT(emp_name)  
FROM employee  
GROUP BY emp_city;
```

The practical showed an error when emp_city was selected but the query grouped by emp_department.

7) WHERE / GROUP BY / HAVING order

```
SELECT ...  
FROM ...  
WHERE ...  
GROUP BY ...  
HAVING ...  
ORDER BY ...;
```

Aggregate conditions such as COUNT(...) > 2 belong in HAVING, not WHERE.

8) Comma matters in SELECT

```
SELECT emp_department, AVG(emp_salary) AS avg_salary  
FROM employee  
GROUP BY emp_department;
```

The practical showed a syntax error when the comma between emp_department and AVG(emp_salary) was missing.

9. Quick Syntax Cheat Sheet

Purpose	Syntax / Example
All rows/columns	SELECT * FROM employee;
Filter	WHERE condition
AND	condition1 AND condition2
OR	condition1 OR condition2
Range	emp_salary BETWEEN 40000 AND 60000
Multiple values	emp_department IN ('IT','HR')
Pattern starts	emp_name LIKE 'A%'
Pattern ends	emp_name LIKE '%a'
One character	emp_name LIKE 'A_'
NULL	emp_salary IS NULL
Not NULL	emp_salary IS NOT NULL
Ascending	ORDER BY emp_salary ASC
Descending	ORDER BY emp_salary DESC

Purpose	Syntax / Example
Count	COUNT(*) / COUNT(emp_name)
Total	SUM(emp_salary)
Average	AVG(emp_salary)
Maximum	MAX(emp_salary)
Minimum	MIN(emp_salary)
Group	GROUP BY emp_department
Group filter	HAVING COUNT(*) > 2
Limit result	LIMIT 1

10. Final Revision Checklist

Before running a query, check:

- ✓ Correct database selected (USE rrgi;)
- ✓ Correct table name (employee)
- ✓ Correct column name (emp_salary, not salary)
- ✓ Text values inside quotes
- ✓ SQL statement ends with ;
- ✓ UPDATE/DELETE has the intended WHERE condition
- ✓ Pattern search uses LIKE
- ✓ NULL uses IS NULL / IS NOT NULL
- ✓ Aggregate filtering uses HAVING
- ✓ Selected non-aggregate column matches GROUP BY

Source basis

This PDF is based on the supplied Assignment 5 question sheet, the supplied corrected Assignment 5 solution, and the project-practice command/output material. The question sheet contains the original 35-task set; the corrected solution contains the cleaned practical versions and correction list.